

Embedded AI Engineering Development Kit

Technical Documentation

The master reference — architecture, every design decision, and how to use every feature.

Audience: firmware engineers (beginner to mid-level), technical writers, contributors.

Reason first. Build second. Verify always.

The Embedded AI Engineering Development Kit

This is the master reference for EAEDK. It explains what the system is, the problem it solves, how it is built, every significant design decision and the reasoning behind it, and how to use every feature. It assumes no prior knowledge of the project. Where the system has a limitation, this document says so and explains why the limitation exists.

This is a living technical reference, not a sales document. If a sentence here ever disagrees with the code, the code is correct and this document has drifted — please open an issue.

Audience: firmware engineers from beginner to mid-level, technical writers, and contributors.

A note on accuracy: every command, rule name, table, file path, and number in this document was taken from the source tree, not from memory. The validation engine has 22 rules (it began with 18); there are 4 risk rules, 15 log signatures, 14 seeded boards, 8 templates, and 14 golden eval cases. Where the document quotes an exact value (an alignment, a margin, a scoring weight), that value is the one in the code.

Table of contents

1. What EAEDK Is (and Is Not)
2. Architecture
3. The Validation Engine in Depth
4. The Knowledge Database
5. The Log Analysis Engine
6. Getting Started
7. CLI Reference
8. Design Decisions and Trade-offs
9. Testing and Validation
10. Limitations and Known Gaps
11. Roadmap
12. Contributing

Chapter 1 — What EAEDK Is (and Is Not)

1.1 The problem it solves

Firmware is the software that runs directly on a microcontroller or processor, with little or no operating system underneath it. Writing firmware is different from writing a normal program because the code has to match the exact hardware it runs on: the right memory addresses, the right clock speed, the right way to turn each peripheral on. Get one of those wrong and the board does not "throw an error" — it simply does nothing, or resets in a loop, or appears dead.

The hardest part of starting firmware is not writing code. It is knowing **what questions to ask before you write any code at all**. This is the "I don't know what I don't know" problem, and it is the specific thing EAEDK exists to address.

A concrete example. A beginner buys an STM32F103 "Blue Pill" — a popular two-dollar board built around an STMicroelectronics Cortex-M3 chip. They want to blink the on-board LED. They search YouTube, find a tutorial, copy the code, change a pin number to match their board, build it, flash it... and nothing happens. The LED stays dark. There is no error message. The beginner has no idea whether the problem is:

- the **clock**: on an STM32, every peripheral is switched off at reset to save power. You must explicitly enable the clock to the GPIO port *before* you touch any of its pins. Skip this and the pin silently does nothing.
- the **boot pins**: the STM32 has a pin called BOOT0 that decides where the chip starts executing. If it is in the wrong position, the chip never runs your program from flash.
- the **vector table**: the table of interrupt handler addresses must sit at the very start of flash memory and be correctly aligned, or the first interrupt crashes the chip.
- the **flash address**: the code must be linked to run from the address where the chip's flash actually lives (`0x08000000` on this chip), not some default.
- the **build target**: if the compiler was told to build for a PC instead of for an ARM Cortex-M3, the resulting file will not run on the chip at all.

An experienced engineer asks all of these questions automatically. A beginner does not know the questions exist. They copy code, modify it blindly, and when it fails they have no map of what could be wrong. This is where most people give up on firmware.

The same problem exists higher up the experience ladder, just with bigger words. A mid-level engineer bringing up Linux on a new board has to get DDR memory timing right before the system will boot stably; has to place the kernel and device-tree blob at non-conflicting addresses in RAM; has to make sure the serial console is configured

or they will have no way to see what went wrong. The questions are different but the structure is identical: a long list of things that must be correct *before* the first boot, most of which produce silent failure if wrong.

EAEDK's job is to know that list, ask those questions for you, check the answers it can check deterministically, and be honest about the answers it cannot check.

1.2 What EAEDK is

EAEDK is a **local-first, offline embedded engineering validation and mentoring platform**. Two phrases in that sentence carry weight:

- **Local-first / offline:** everything runs on your own computer. The knowledge lives in a single database file. There is no cloud service, no account, no per-request cost, and it works with the network unplugged. (The one optional component — a local language model — also runs on your machine; see Chapter 2.5.)
- **Validation and mentoring:** EAEDK checks engineering decisions against hardware facts (validation) and teaches a beginner the order to learn things and the questions to ask (mentoring).

Three things EAEDK deliberately is **not**, internally:

- **It is not a code generator first.** It does generate real, buildable scaffold files (Chapter 2.3, output engine), but the generated code is a starting point with the hardware facts filled in correctly — not the product. The product is the checking and the teaching.
- **It is not a chatbot.** There is an optional language model, but it is a thin convenience layer, not the core.
- **It is a system where the deterministic engines hold the truth and the language model only explains.** This inversion — described in detail in 1.4 — is the single most important design idea in the project.

1.3 What EAEDK is not

Being explicit about boundaries is more useful than a feature list. EAEDK is **not**:

- **Not a replacement for reading datasheets.** A datasheet is the manufacturer's official document describing a chip. EAEDK can extract some facts from a datasheet PDF and tell you where to look for the rest, but it cannot replace your reading the relevant chapters. It is designed to make you faster at reading the *right* parts, not to let you skip them.
- **Not a guarantee of correct firmware.** EAEDK checks the specific things it has rules for (memory fit, address conflicts, alignment, toolchain match, and so on). A project that passes every EAEDK check can still have bugs EAEDK has no rule for. "Feasible" means "none of the checks I can run found a blocker," not "this will work."
- **Not a cloud service.** There is no server to sign up for. This is a constraint EAEDK treats as a feature (Chapter 8).

- **Not a general-purpose AI assistant.** The optional language model is constrained to explain EAEDK's own results and is forbidden — structurally, not just by instruction — from asserting hardware facts (Chapter 2.5).
- **Not a debugger or a flashing tool.** EAEDK tells you which tools to install and generates the commands to run, but it does not itself talk to your board over USB or a debug probe.

1.4 The core philosophy

EAEDK is built around one rule:

The language model may explain, draft, and triage. It may never assert a hardware fact that the database has not verified.

To see why this matters, you need the **trust hierarchy** — the order in which EAEDK trusts information:

1. **Datasheet facts** (highest trust): a value taken from the manufacturer's document, with a citation to the page and section, confirmed by a human.
2. **Validated engineer inputs**: a value you typed in (your firmware's size, your chosen memory layout), which the engines can check for internal consistency.
3. **Language-model reasoning** (lowest trust): plausible-sounding text that may be wrong.

The order matters because of what is at stake. Every general-purpose AI coding assistant will happily tell you that "the STM32F407 runs at 168 MHz," or invent a DDR timing, or guess a register address — confidently, and sometimes wrong. On a normal program a wrong constant is a bug you find and fix. On hardware, a wrong clock value means every delay and every serial baud rate is wrong; a wrong register address means you write into memory you did not mean to; a wrong DDR timing means memory corrupts intermittently in ways that take weeks to track down. A confident wrong answer can cost you a board, or worse, a month.

So EAEDK inverts the usual arrangement. In a typical AI tool, the model is the source of answers and guardrails try to catch its mistakes afterward. In EAEDK, **deterministic engines are the source of answers**, and the model is an optional layer on top that can only rephrase and explain what the engines already established.

The enforcement mechanism is the **post-filter** (Chapter 2.5). When the language model produces text, EAEDK builds an *allowlist* of numbers that are actually cited — the board's real flash and RAM values from the database, the values you typed in, and numbers quoted verbatim from a document being analyzed. Then it scans the model's output sentence by sentence. Any sentence that contains a hardware number — a hexadecimal address, a memory size, a clock frequency, a timing — that is **not** in the allowlist is removed entirely and replaced with a marker: `[uncited claim removed – verify against TRM]`. (TRM = Technical Reference Manual, the detailed companion to a datasheet.)

The post-filter is deliberately blunt. It removes the *whole sentence*, not just the number, because a sentence built around an invented value is unsafe even if the surrounding words are fine. It treats *all* clock frequencies and timing values as uncited by default, because those values are not stored in the database at all — so if the model emits one, it was invented. The design accepts that this will sometimes remove a sentence that happened to be true. That trade is intentional: dropping a true-but-uncited claim is cheaper than letting an invented one reach an engineer who will trust it.

1.5 Who it is for

EAEDK is built for a range of experience levels. Each gets something different, and for each there is a point where EAEDK's value ends and your own engineering begins.

Level	Who	What EAEDK gives them	Where EAEDK's value ends
Level 0	Complete beginner. No computer-science background, no hardware experience.	The ordered list of what to learn first; the "think before you code" questions for their exact board; a path to a blinking LED in a free online simulator with no hardware to buy.	The moment they need to design original hardware or write a non-trivial algorithm — EAEDK checks bring-up, not application logic.
Level 1	Hobbyist. Some Arduino or ESP32 experience.	The jump from "copy a sketch" to understanding clocks, memory layout, and why their code is structured the way it is; honest answers about a board they have not used.	Advanced peripheral work (DMA, complex timer modes) where EAEDK has no specific rule — it will not stop you, but it cannot check you.
Level 2	Junior firmware engineer, 0–2 years.	A pre-flight checklist that catches the classic mistakes (image too big for flash, bootloader and app regions overlapping, wrong toolchain) before they burn a day; cited facts to put in code review.	Domain-specific correctness (a motor-control loop, a USB stack) — outside EAEDK's rule set.
Level 3	Mid-level engineer doing Linux bring-up and driver development.	DDR-timing-verified gating (the system refuses to call a board "ready" while DDR timing is unconfirmed); kernel/DTB load-address conflict checks; project-aware log triage that correlates a boot failure against the project's own unverified assumptions.	Deep silicon-specific debugging and original driver logic — EAEDK frames the problem and checks the structure, but the engineering is yours.

The common thread: EAEDK is strongest at the **start** of a project (what to check before the first boot) and at the **failure** of a boot (what a log is telling you). It is weakest, by design, in the middle — the part that is genuinely your engineering work.

Chapter 2 — Architecture

2.1 The two interfaces

EAEDK has two front doors, and they call exactly the same engine functions underneath.

The CLI (command-line interface) is the primary interface. You type commands like `eaedk validate myproject` into a terminal. It is built on `argparse`, which is part of Python's standard library, so the CLI itself adds no dependencies. The whole offline tool needs only one third-party package, `PyYAML`, to read its data files. (The optional language model and the optional PDF reader add dependencies only if you choose to install them.)

***Documented deviation — `argparse` instead of `Typer`.** `Typer` is a popular library for building nicer-looking CLIs. EAEDK uses plain `argparse` instead. The practical reason: during early development the environment for installing extra packages was unreliable, and a tool whose entire selling point is "works offline, single dependency" should not depend on a CLI framework to start up. `argparse` is in the standard library and cannot fail to install. It stayed because the constraint it satisfies — minimal dependencies — is permanent, not a temporary inconvenience.*

The Web UI is an optional browser interface. You install it with `pip install -e '[web]'` and start it with `eaedk web`; it opens at `http://localhost:8080`. It is built on FastAPI (a Python web framework) serving plain, hand-written HTML pages — `boards`, `setup`, `validate`, `export`, `studio`, `ingest` (the datasheet tab), `logs`, and `mentor`.

***Documented deviation — plain HTML instead of `React`.** `React` (and similar frameworks) would mean a build step, a package manager, and a large dependency tree — all of which fight the offline-first goal. The Web UI is static HTML files plus one shared JavaScript file. There is no build step. The cost is that the UI is plainer than a modern web app; the benefit is that it has no toolchain of its own and works wherever Python does.*

Why two interfaces, and why they must share one engine. The CLI suits engineers and scripting; the Web UI suits beginners who are more comfortable clicking than typing. The hard rule is that the Web UI must never have its own copy of any logic. Look at the Web server (`web/server.py`): its route handlers are thin wrappers that call the same functions the CLI calls — `assess_project`, `export_project`, `analyze_log`, `intelligence_report`, `answer_query`. This is enforced by convention and by code review (Chapter 12): duplicating engine logic into the Web layer is explicitly forbidden. The reason is correctness. If validation logic lived in two places, the two interfaces would eventually disagree, and a beginner on the Web UI would get a different answer than an engineer on the CLI for the same project. One engine, two skins.

2.2 The trust boundary

The single most important diagram in EAEDK is the line between what is trusted and what is filtered.



What "inside the boundary" means: these components only ever state values that came from the database, from a citation, or from an engineer's typed input. A validation rule cannot make up a flash size — if the size is missing, the rule returns `UNKNOWN`, not a guess. The orchestrator assembles answers only from these sources.

What "outside the boundary" means: the language model can produce any text at all, including invented numbers. So everything it produces is passed through the post-filter before it reaches you. The boundary is the post-filter: facts cross from inside to outside as a numeric allowlist, and text crosses from outside to inside only after every uncited hardware number has been stripped.

2.3 The deterministic engine layer

These are the engines that hold the truth. Each is described in more depth in later chapters; this is the map.

Validation Engine (`engines/validation/rules.py`). 22 rules, each a pure function that takes a context dictionary and returns one of three results: **PASS** (the check succeeded), **FAIL** (the check found a real problem), or **UNKNOWN** (there is not enough verified data to decide). A "pure function" means it has no side effects — same input, same output, no database writes, no network. Each rule carries a **severity** (`HIGH`, `MEDIUM`, or `LOW`) and a **gating** flag that says whether it can block feasibility. The engine rolls the individual results up into an overall feasibility verdict (Chapter 3.4). Rules never call the language model and never invent a missing value.

Risk Engine (`engines/risk/engine.py`). Where validation rules answer yes/no/unknown, risk rules flag *concerns* — situations that are not outright failures but deserve a warning, like "your image uses more than 90% of flash." Risk rules are **data-driven**: the conditions are written in a tiny domain-specific language (DSL) stored as text, not as Python code. There are two kinds of risk in the system: ephemeral **risk-engine findings** that are recomputed every time you run an assessment, and persistent **tracked risks** that get written into the database when log triage implicates a project gap (Chapter 5.4). The distinction matters and is covered in Chapter 8.

Toolchain Engine (`engines/toolchain/`). The build environment is treated as a first-class thing to validate. `toolchain detect` inventories the compilers, debuggers, flash tools, and build systems on your computer. `toolchain validate` cross-references what you have against what the board needs — for example, a board with an ARM Cortex-M chip needs `arm-none-eabi-gcc`; if your computer only has the regular PC compiler, the project is not feasible, and the engine produces a one-line fix (`apt install gcc-arm-none-eabi`). The teach strings are concrete install commands, not vague advice.

Log Engine (`engines/logs/`). Analyzes boot logs and crash dumps. It detects the log format (U-Boot bootloader output, Linux kernel `dmesg`, bare-metal MCU crash, or a completely silent boot), matches the text against a database of known failure signatures, and — only when nothing matches — slices a window around the crash and hands it to the post-filtered language model for structural hypotheses. Findings can be written back into a project as tracked risks. Covered fully in Chapter 5.

Output Engine (`engines/output/`). Turns a validated project into real files you can build: a checklist, a CMake build configuration, a linker script with the board's real memory addresses filled in, a starter `main.c`, a `START_HERE.md` guide, and optionally Wokwi simulator files. It is **gated on feasibility**: a project that is not feasible is refused by default (you can force a clearly-labeled DRAFT). The `START_HERE.md` guide offers two paths — simulate first in Wokwi (no hardware needed), then flash to a real board — and the simulation path comes first deliberately (Chapter 8).

2.4 The knowledge database

All of EAEDK's knowledge lives in one SQLite database file at `~/.eaedk/eaedk.db`. SQLite is a small, serverless database that is a single file on disk — no separate database process, nothing to install or configure.

The schema is built by forward-only migrations. A migration is a numbered SQL script that changes the database structure. EAEDK has twelve, `0001` through `0012`. When you run `eaedk db init`, the system checks which migrations have already been applied (using SQLite's `user_version` counter) and runs any that are newer. There are **no rollback scripts** — migrations only go forward. The reasoning is in Chapter 8, but in short: a rollback that runs against a real user's database risks destroying their data, and "undo" is better handled by a new forward migration that fixes the previous one.

The `engineering_facts` VIEW. A database VIEW is a saved query that looks like a table but is computed on demand. The `facts` table stores individual hardware facts; the `engineering_facts` view joins each fact to its citation (page, section, snippet) and its source document, so that anything reading a fact also gets its provenance without having to do the joins itself. This is the read interface the post-filter and the report engine use. Why a view instead of just storing everything flat in one table? Because provenance must never be flattened away — a fact without its citation is exactly the kind of free-floating number EAEDK is built to distrust. The view keeps identity (in typed `boards` columns) and provenance (in `sources / citations`) properly separated while presenting a convenient unified read.

Seed data lives in YAML, not SQL. YAML is a plain-text format that is easy to read and easy to diff (compare line by line in version control). All of EAEDK's seed knowledge — boards, templates, risk rules, log signatures, capabilities, learning paths — lives in `packages/` as YAML files. `eaedk db seed` reads those files into the database. Why not just write SQL `INSERT` statements? Because a board profile expressed as YAML is something a contributor can read, review in a pull request, and edit without knowing SQL, and a reviewer can see exactly what changed line by line. Facts that matter to humans should live in a human-readable, diffable form. Chapter 4 documents the schema and the YAML formats in full.

2.5 The LLM layer

LLM stands for "large language model" — the kind of AI that produces human-like text. EAEDK's use of it is deliberately minimal and tightly fenced.

Ollama and the default model. EAEDK talks to a local model through [Ollama](#), a program that runs language models on your own machine. The default model is `qwen2.5-coder:3b` — a small (3-billion-parameter) coding model. EAEDK reaches Ollama at `http://localhost:11434` by default; both the host and the model are overridable through environment variables (`EAEDK_OLLAMA_HOST` , `EAEDK_LLM_MODEL`). Everything runs locally; nothing is sent to a cloud.

The model is off by default. Every command that can use the model requires the `--llm` flag to turn it on; without the flag you get the deterministic answer only. This is the opposite of most AI tools, where the model is always on. The reason is the philosophy of 1.4: the deterministic answer is the product, and the model is an optional convenience. A user should be able to do real work having never started Ollama at all.

Graceful degradation. If you pass `--llm` but Ollama is not running, or the model is not downloaded, EAEDK does not crash and does not hide it. It prints a plain message — for example, that the gateway is unavailable and that you can run `ollama pull qwen2.5-coder:3b` — and the deterministic assessment above it is unaffected. The model failing is always a non-event for the core.

The gateway and constrained prompts. The `Gateway` (`llm/gateway.py`) is the single entry point to the model. It runs the deterministic assessment first, then sends the model only the *already-cited context* plus your question, with a system prompt (`llm/prompts.py`) that states the hard rules: explain the engines' results, never state a hardware fact unless it appears verbatim in the provided context, never invent register addresses or DDR timings or clock values, and surface anything marked UNKNOWN as UNKNOWN. The prompt is "belt and suspenders" — useful, but not the real enforcement.

The post-filter is the real enforcement (`llm/postfilter.py`). After the model responds, the gateway builds an allowlist of cited numbers (board geometry, human-verified facts, your project inputs) and runs `filter_text` over the model's output. The filter splits the text into sentences and, for each sentence, checks every hexadecimal literal, every memory size (with both binary and decimal interpretations of KB/MB/GB), and treats every frequency (kHz/MHz/GHz) and timing (ns/μs/ms) as automatically uncited. Any sentence containing a hardware number not in the allowlist is replaced with the removed-claim marker. The number of removed claims is reported to you. This is structural: even if the model ignores every instruction in the prompt, an invented address physically cannot reach you.

The Actor-Critic loop (`actor_critic.py`). This is the most elaborate use of the model, used by `mentor --review-code` . It reviews a beginner's firmware scaffold for common mistakes using two roles played by the *same* model instance, time-sliced (the model is called with a "critic" system prompt, then with an "actor" system prompt):

- The **Critic** reviews the code for beginner mistakes — a peripheral used before its clock is enabled, wrong initialization order, a stack too small or larger than available RAM, a buffer larger than RAM, overlapping OTA partition slots, a missing device-tree compatible string. It returns structured issues. It is forbidden from inventing hardware values.
- The **Arbiter** is deterministic, not a model. When the Critic raises a memory concern, the Arbiter re-checks it with the *real* `RAM_BUDGET` validation rule against the board's verified RAM. Only if the real rule returns FAIL is the issue marked **CONFIRMED**; otherwise it is downgraded to **Advisory**. So the agents propose, and the engine decides.
- The **Actor** explains how to fix the confirmed issues. Its text goes through the same post-filter as everything else.

The CONFIRMED issues always start from `grounded_confirmations` — the actual failures from the Validation Engine over the project's real inputs — so the deterministic faults are reported even if the model says nothing useful. The loop runs at most **2 epochs** (passes). Why one time-sliced model instead of two separate models? Chapter 8 — in short, to keep the offline footprint to a single downloaded model.

2.6 The mentor layer

The mentor layer teaches. It is mostly deterministic; the model is an optional explainer on top.

Board capability maps. Each board records what it can do — `gpio`, `timer`, `uart`, `spi`, `i2c`, `usb`, and so on — as capabilities, each with a one-sentence plain-language summary stored in the database. `eaedk mentor --board <name>` prints these so a beginner knows what the board is actually for.

Learning paths, and why order matters. EAEDK stores an ordered learning path — a sequence of projects to build in deliberate order (blink, then UART, then a bootloader, and so on). The order is not arbitrary: each step lists the capabilities the board must have and a reason it comes before the next. The path is **filtered to what the board can actually do** — a board without USB does not show the USB step. Crucially, a step that is filtered out is never silently dropped: the mentor shows it under "steps not shown yet" with the exact command to unlock it (`eaedk board capability add ...`). A beginner should always learn *why* something is hidden, never just find it missing.

Think-before-code checklists, generated from facts. `eaedk mentor --board <name> --think` produces the questions to answer before writing code — but these are **generated from the board's verified facts**, not hardcoded per board. The function reads the SoC family, the memory geometry, and the capabilities, plus any seeded board-specific "blink facts" (the on-board LED pin, its clock domain). For an STM32 it asks about enabling the GPIO clock in RCC; for an RP2040 it asks about the second-stage bootloader; for an ESP32 it asks about the partition table; for an AVR it asks about `F_CPU` and fuses. Where a fact is not in the database, the hint points you to where to find it rather than asserting a value. This is the trust hierarchy applied to teaching.

The driver development path. Boards whose SoC is an application processor (an ARM Cortex-A core that can run Linux) get an additional, separate learning path for writing Linux device drivers. This path only appears for boards that actually run Linux — determined deterministically from the architecture string.

The Engineering State Engine (`engines/state.py`). This answers "how far along is my project?" — and it answers it from **evidence**, never from a stored number and never from the model. Each checklist item's status is derived in priority order: COMPLETE if its validation rule(s) all PASS (verified by the engine), or if its rule is a resolved tracked risk (resolved from a log), or if you explicitly confirmed it (`eaedk checklist done`); IN_PROGRESS if you have provided values but they are not yet verified; NOT_STARTED otherwise. The percentage is computed (complete ÷ total) every time, never persisted. This is the trust hierarchy applied to progress: the system reports what it can prove, attributes each completion to how it was proven, and the model is not allowed to move the needle.

The conversational mentor (`mentor_llm.py`). `eaedk mentor --board <name> --chat` opens a back-and-forth conversation. Each reply injects the board's facts and the project's real progress (from the State Engine) into the model's context, ties advice to that specific board, knows whether you are simulating in Wokwi or on real hardware, and is post-filtered like every other model output. Ask "how am I doing?" and it answers from the State Engine, not from guesswork.

2.7 The datasheet intelligence engine

This engine (`engines/ingest/`) reads a datasheet PDF and turns it into cited, reviewable facts. Its guiding rule: extracted facts are **staged**, never trusted automatically. A human confirms each one before it becomes truth.

The ingestion pipeline. 1. **Format detection and text extraction** (`pdf.py`): the PDF is read with PyMuPDF (a PDF library, imported only if installed) into a list of pages of text. 2. **Sentence-aware fact extraction** (`extract.py`): each page is split into sentences. For every hexadecimal value the extractor finds the *nearest* memory keyword (`Flash` , `SRAM` , `RAM`) by distance and assigns the address to it, so a flash address is not mislabeled as a RAM address. Sizes are matched to the nearest memory keyword and converted to bytes (`512 KB` → 524288, `64 Kbytes` → 65536). A maximum clock is read from phrases like "up to 72 MHz." 3. **Confidence assignment**: a value found in a **table** (a short line of label + hex) is marked **HIGH**; a value found in **prose** (a full sentence) is marked **MEDIUM**. Prose is *never* upgraded to HIGH. This rule reflects reality: a number in a memory-map table is far less likely to be a misreading than a number pulled from a sentence. 4. **Staging**: every extracted value lands in the `fact_candidates` table with status `pending` . Nothing is written to the trusted `facts` table until a human confirms it with `eaedk ingest --board <name> --review` then `--confirm <id>` .

The intelligence report (`report.py`) — seven deterministic sections, no model involved: 1. **What I found** — the extracted facts, each cited and confidence-rated, with human-readable labels. 2. **What I could not find** — mandatory bring-up items that were not extracted (boot pins, watchdog state, power-on reset), each with *why* it is

mandatory and *where in the datasheet to look*. 3. **Priority order** — the bring-up items ranked, found or not. 4. **Risk warnings** — architecture-specific risks for this chip family. 5. **Closest known board** — the most similar board EAEDK already knows (see below). 6. **What EAEDK can and cannot do** — an honest split based on what was actually extracted. 7. **Your immediate next step** — the single most important thing to find next.

The board similarity engine (`repo.find_similar_boards`). Scoring is deterministic, with exact weights: **+40** for the same architecture, **+20** if flash size is within 2× of the query board, **+20** if RAM is within 2×, **+10** for the same vendor, **+10** for two or more shared peripherals. The total maps to a confidence band: **HIGH** at 70 or above, **MEDIUM** at 40–69, **LOW** below 40. A match is used to suggest "this will probably work the same" (clock setup patterns, register layout for the family) and "you MUST verify these from your own datasheet" (exact register addresses, the pin alternate-function table) — never to assume values. For a freshly-ingested board whose geometry is still in pending candidates, the similarity engine reads those pending values so the match still scores, flagged as unconfirmed.

The query engine (`query.py`) — `eaedk ask --board <name> "question"` . It answers in one of several modes, always with an explicit confidence and always ending in an action: - **HIGH, cited** when the fact is in the database (geometry or a confirmed datasheet fact), with the page/section and what the value means for your code. - **UNKNOWN** for a mandatory-but-missing item (boot pins, watchdog, power-on reset), with the exact search terms and a hard "do not proceed until confirmed" warning. - **LOW, "ingested but not extracted"** when a datasheet was ingested but this particular value was not pulled out — with the search terms and the `--review` instruction. - **LOW, "no datasheet ingested"** when there is nothing to answer from, pointing you at `eaedk ingest` . - **MEDIUM** when the model elaborates on general behavior — post-filtered, with a "verify before you write code" caveat.

The two top-level modes are simply "we have a datasheet for this board" versus "we do not," and the engine never blurs them, because telling a beginner "I couldn't find it" when no datasheet was ever loaded is a different and more honest message than implying the datasheet lacked it.

Chapter 3 — The Validation Engine in Depth

3.1 Rule anatomy

Every validation rule is a small Python function decorated with `@rule(...)`. The decorator and the result together define these fields:

- **name** (e.g. `FLASH_CAPACITY`): the stable identifier, used in templates, eval cases, and output.
- **goals**: which project goal types the rule applies to (e.g. `bootloader`, `ota`), or "all goals." A rule for OTA partition layout simply does not run for a bare-metal blink project.
- **user_inputs**: the engineer-provided keys the rule reads. These drive the "engaged" concept (below).
- **PASS / FAIL / UNKNOWN conditions**: the logic. PASS = the check succeeded. FAIL = a real, specific problem was found. UNKNOWN = there is not enough verified data to decide.
- **severity** (`HIGH` / `MEDIUM` / `LOW`): how serious a failure is.
- **gating**: whether this result can block the overall feasibility verdict. Defaults to true; the toolchain engine sets it false for non-critical tools.
- **teach string**: a one-line, plain-language explanation attached to any non-PASS result — what the field is, its units, where to find it, and the consequence of getting it wrong. This is what keeps a beginner from being left with a bare rule name.

A rule is "**engaged**" when you have supplied at least one of its `user_inputs` (or the rule needs none). This distinguishes "you have not started this yet" (surfaced gently as missing information) from "you started it but the values do not check out" (which blocks). An UNKNOWN on a rule you have not engaged is just a to-do; an UNKNOWN on a rule you *have* engaged is a blocker.

3.2 The complete rule catalog

All 22 rules. "Gating" means a HIGH-severity failure or engaged-unknown blocks feasibility.

Rule	What it checks	Key inputs	Severity	Goals
FLASH_CAPACITY	Firmware image fits in flash (with a 10% reserve band note)	estimated_image_size , board flash size	HIGH	all
RAM_BUDGET	stack + heap + statics fit in RAM (10% reserve band)	stack_size , heap_size , static_size , board RAM	HIGH	all
VECTOR_TABLE_PLACEMENT	Interrupt vector table is inside flash and correctly aligned	vector_table_addr	HIGH	bootloader, bare_metal_app
BOOTLOADER_APP_NO_OVERLAP	Bootloader and application flash regions do not overlap and fit in flash	bl_region , app_region	HIGH	bootloader, ota
PARTITION_LAYOUT_FITS	All partitions fit within storage	partitions , primary_storage_bytes	HIGH	ota, linux
PARTITION_NO_OVERLAP	No two partitions overlap	partitions	HIGH	ota, linux
PARTITION_AB_SYMMETRY	OTA A/B update slots are equal in size	partitions	MEDIUM	ota
RECOVERY_PRESENT	A recovery (or slot_b) partition exists for fail-safe update	partitions	HIGH	ota
LOAD_ADDR_CONFLICT	Kernel and DTB load addresses are in DDR, clear of the init region, and distinct	kernel_load_addr , dtb_load_addr , ddr_base , ddr_bytes	HIGH	uboot, linux
DDR_TIMING_VERIFIED	DDR memory timing has been confirmed from the datasheet	ddr_timing_verified	HIGH	uboot
BOOT_FLOW_CONSISTENCY	bootloader → kernel → init addresses are all distinct	bootloader_load_addr , kernel_load_addr , init_addr	HIGH	uboot, linux
CONSOLE_UART_DEFINED	A debug console UART is defined (and stdout-path for Linux)	console_uart	MEDIUM	uboot, linux, bare_metal_app
TOOLCHAIN_ARCH_MATCH	The compiler's target architecture matches	toolchain_arch	HIGH	all

Rule	What it checks	Key inputs	Severity	Goals
	the chip			
<code>SDK_HOST_OS_MATCH</code>	The host OS satisfies any SDK requirement	<code>host_os</code>	MEDIUM	all
<code>DRIVER_COMPATIBLE_STRING</code>	A Linux driver's device-tree compatible string is set	<code>dtb_compatible</code>	MEDIUM	driver
<code>REGISTER_MAP_PRESENT</code>	A human-verified register map exists for driver work	<code>register_facts</code>	HIGH	driver
<code>PINMUX_CONFLICT</code>	No physical pin is assigned to two signals	<code>pin_assignments</code>	HIGH	all
<code>POWER_SEQUENCE</code>	Power rails come up in a valid, unambiguous order	<code>power_rails</code>	HIGH	all
<code>SECURE_BOOT_SIGNATURE_VERIFY</code>	Bootloader verifies the image signature before running it	<code>secure_boot_sig_verify</code>	HIGH	bootloader, ota
<code>SECURE_BOOT_KEY_STORAGE</code>	The verification key lives in immutable storage (eFuse/OTP/TPM/...)	<code>secure_boot_key_storage</code>	HIGH	bootloader, ota
<code>SECURE_BOOT_ROLLBACK_COUNTER</code>	A monotonic anti-rollback counter is present	<code>secure_boot_rollback_counter</code>	HIGH	bootloader, ota
<code>SECURE_BOOT_DEBUG_LOCKED</code>	The debug interface is locked in the production build	<code>secure_boot_debug_locked</code>	HIGH	bootloader, ota

What goes wrong in the real world if each is ignored:

- **FLASH_CAPACITY** — the firmware compiles but does not fit; flashing fails or silently truncates. The 10% reserve band warns you before you are dangerously close, because flash usage tends to grow as a project matures.
- **RAM_BUDGET** — stack, heap, and static data collide at runtime. There is no compile error; the board crashes unpredictably, often only under load, which is the hardest kind of bug to find.
- **VECTOR_TABLE_PLACEMENT** — the table of interrupt handler addresses must sit at the start of flash and be aligned (256 bytes on Cortex-M0/M0+, 512 on M3/M4/M7). Misplaced, the chip jumps to a garbage

address on the first interrupt and faults.

- **BOOTLOADER_APP_NO_OVERLAP** — flashing the application corrupts the bootloader (or vice-versa); the device bricks on the next reset.
- **PARTITION_LAYOUT_FITS / PARTITION_NO_OVERLAP** — partitions that exceed storage will not flash; partitions that overlap corrupt each other's data.
- **PARTITION_AB_SYMMETRY** — in an A/B update scheme both slots must hold a full image; if one is smaller, an update that fits slot A may not fit slot B and fail unpredictably.
- **RECOVERY_PRESENT** — without a recovery partition, a failed over-the-air update bricks the device with no way back.
- **LOAD_ADDR_CONFLICT** — a kernel or device-tree blob loaded over the memory the bootloader is still using corrupts early boot; the system hangs before it can tell you why.
- **DDR_TIMING_VERIFIED** — external DDR memory needs precise timing parameters; unverified ones cause intermittent corruption that looks like random crashes. This rule is the canonical example of why UNKNOWN must block (3.3).
- **BOOT_FLOW_CONSISTENCY** — overlapping handoff addresses between bootloader, kernel, and init mean one stage overwrites the next.
- **CONSOLE_UART_DEFINED** — without a debug console you have no log, so a failed boot is a black box. MEDIUM, not HIGH, because you *can* technically boot without it — you just cannot debug.
- **TOOLCHAIN_ARCH_MATCH** — a compiler targeting the wrong architecture produces a binary the chip cannot execute (or will not build at all).
- **PINMUX_CONFLICT** — two peripherals wired to the same physical pin means one of them silently does not work.
- **POWER_SEQUENCE** — powering rails up in the wrong order can hang the chip or, on some silicon, damage it.
- **SECURE_BOOT_*** — the four secure-boot rules guard the chain that stops unsigned or rolled-back firmware from running and stops an attacker reading firmware out through an open debug port. Each engages only on its own input key, so they never affect a project that is not doing secure boot.

3.3 UNKNOWN vs FAIL

This was the hardest design decision in the project, and getting it right is what separates EAEDK from a tool that gives confident wrong answers.

UNKNOWN is not a soft pass. A natural temptation when a value is missing is to treat the check as "probably fine" and move on. EAEDK refuses to. If a HIGH-severity gating rule returns UNKNOWN *and the rule is*

engaged, the whole project is **blocked** — not feasible — until the value is supplied.

The canonical case is DDR timing. When `ddr_timing_verified` is not set, `DDR_TIMING_VERIFIED` returns UNKNOWN, and a U-Boot project is reported as **blocked**, not feasible. The principle is "**unverified DDR timing = blocked.**" A beginner might object: "but I have not said it is wrong, only that I have not checked it." That is exactly the point. On hardware, "I have not verified this" and "this is safe" are not the same statement, and treating them as the same is how boards get corrupted memory and engineers lose weeks. EAEDK encodes the discipline a senior engineer applies automatically: *you do not get to call it ready while a critical value is unconfirmed.*

The debate was whether this would frustrate beginners by blocking projects that "felt" fine. The resolution was the **engaged** concept (3.1): an UNKNOWN on a rule you have not started is surfaced quietly as missing information, not as a blocker. Only once you have *engaged* a rule — supplied some of its inputs — does an UNKNOWN on it block. This keeps the discipline where it matters (you are actively working on DDR, so you must finish verifying it) without nagging about checks you have not begun.

3.4 Feasibility calculation

The individual PASS/FAIL/UNKNOWN results roll up into one verdict, computed only over **gating** results:

```
any gating FAIL                → not_feasible
else any gating UNKNOWN that is engaged → blocked
else                           → feasible
```

There is also a `no_geometry` verdict produced earlier by the orchestrator when the board has no flash/RAM values at all — without geometry, the memory rules cannot run meaningfully, so the system says so plainly instead of pretending.

The gating flag is what lets some checks inform without blocking. Memory and address rules are gating: a real conflict there must stop you. **Toolchain rules are non-gating by default** for everything except the compiler itself. A missing debugger or build system is reported with full PASS/FAIL/UNKNOWN detail and a fix, but it does not declare your whole design infeasible — you can still reason about the project and export a draft while you go install OpenOCD. A missing or wrong-architecture *compiler*, by contrast, is HIGH and gating, because without it there is no firmware at all.

3.5 The eval harness

The eval harness is EAEDK's regression safety net. ("Regression" = a change that breaks something that used to work.)

Golden eval cases are fixed scenarios with known correct answers, stored as YAML (`packages/knowledge-seed/eval_cases.yaml`) and loaded into the `eval_cases` table. Each case lists a board, a goal type, a set of inputs, and the expected outcome. `eaedk eval run` runs every case through the real engines and checks the result. There are 14 cases today (Chapter 9.2 documents each).

The subset-matching design is what makes the harness robust as the system grows. Each case asserts only a *subset* of the full result — the specific feasibility verdict, the specific rule statuses, the specific risks it cares about (`feasibility` , `validations{key:status}` , `risks_contains` , `risks_absent` , `unknowns_contains`). It does not assert the entire output. This matters because when you add a new rule (say a 23rd), existing cases that never mention it keep passing — the new rule's result is simply not in the subset they check. Without subset matching, every new rule would break every existing case, and the harness would become something contributors route around instead of trust.

To write a new eval case, add an entry to `eval_cases.yaml` with a descriptive name, the goal type, the inputs (including a `board` that exists in the seed), and an `expected` block asserting only what the case is about. Re-seed and run `eaedk eval run --case <name>` . A good case is small and tests one thing — the secure-boot debug-open case asserts only that `SECURE_BOOT_DEBUG_LOCKED` is FAIL and the project is not feasible, nothing more.

Chapter 4 — The Knowledge Database

4.1 Schema reference

The database is built by twelve forward-only migrations. Each table and the migration that added it:

Migration 0001 — initial schema.

- `sources` — every fact's origin document. Columns: `id`, `type` (one of `datasheet`, `trm`, `sdk_doc`, `errata`, `manual`, `web`, `seed`, `user`), `title`, `uri`, `hash`, `created_at`. Exists so every fact can point at where it came from.
- `citations` — a specific location in a source. Columns: `id`, `source_id` (→ `sources`), `page`, `section`, `bbox_json` (bounding box on the page), `snippet` (the quoted text). This is the provenance backbone.
- `socs` — the system-on-chip (the actual processor). Columns: `id`, `name` (unique), `vendor`, `arch`, `notes`. A board is built around a SoC; separating them lets multiple boards share one SoC profile.
- `boards` — a physical board. Columns: `id`, `soc_id` (→ `socs`), `name` (unique), `flash_base`, `flash_bytes`, `ram_base`, `ram_bytes`, `ddr_type`, `ddr_bytes`, `primary_storage`, `boot_modes_json`, `source_id`, `confidence` (default `HIGH`). The typed geometry columns live here (not as loose facts) so validation rules can read them quickly and safely.
- `board_capabilities` — what a board can do. Columns: `id`, `board_id`, `capability`, `details_json`.
- `facts` — individual hardware facts. Columns: `id`, `board_id`, `kind` (`register`, `memmap`, `clock`, `pinmux`, `timing`, `partition`), `key`, `value`, `citation_id`, `confidence`, `verified_by_human` (0/1), `created_at`. Extended by later migrations.
- `templates` and `template_items` — the project checklists (4.3). `templates` is keyed by `(key, version)`; `template_items` holds each checklist item with its required inputs and the validation rule keys it maps to.
- `projects`, `project_inputs`, `project_checklist`, `project_facts`, `decisions`, `risks` — your working project data. `projects` pins a `template_id` (the exact template version in use). `project_inputs` stores values you set, with `source` (`user` / `extracted` / `seed`), citation, and confidence. `risks` records risk findings with a `status` (`open` by default; later migrations add the tracked/resolved lifecycle).
- `risk_rules` — the data-driven risk rules (DSL conditions) seeded from YAML.
- `eval_cases` and `eval_runs` — the golden cases and a log of every run.

Migration 0002 — engineering facts. Adds a `domain` column (`MEMORY`, `CLOCK`, `PINMUX`, ...) and a `source_type` column to `facts`, backfills existing rows, and creates the `engineering_facts` VIEW that joins facts to citations and sources. This is the unified read interface (Chapter 2.4).

Migration 0003 — log engine. Adds `log_signatures` (the known-failure database), `log_files` (analyzed logs, optionally tied to a project), and `log_analyses` (each match or triage result with a confidence).

Migration 0004 — risk resolution. Adds `resolved_at` and `resolution_note` to `risks`, enabling the tracked → resolved lifecycle.

Migration 0005 — toolchain. Adds `toolchain_components` (what `toolchain_detect` found, replaced on each detect) and `board_toolchain_reqs` (the per-board required tool profile, seeded from board YAML).

Migration 0006 — fact candidates. Adds `fact_candidates` — the staging table for datasheet ingestion. Extracted values land here with `method` (`table` / `text` / `llm`), `confidence`, `page`, `section`, `snippet`, and `status` (`pending` / `confirmed` / `rejected`). Nothing reaches `facts` without confirmation.

Migration 0007 — mentor. Adds `capabilities` (capability → plain summary), `learning_steps` (the ordered path with required capabilities and reasons), and `concepts` (concept → one factual anchor sentence).

Migration 0008 — on-ramp. Adds `soc_defaults` (standard flash/RAM geometry per SoC), so a board with a recognized SoC but missing geometry can be filled without a datasheet.

Migration 0009 — last mile. Adds `debug_probes` (probe → OpenOCD interface config) and `soc_flash_profiles` (SoC → OpenOCD target config + the probe a beginner most likely owns), so the generated flash command can be concrete instead of full of placeholders.

Migration 0010 — full coverage. Adds `first_mistakes` (per chip-family beginner mistakes) and `learning_step_intro` (what each next project introduces, cross-linked to a concept).

Migration 0011 — state engine. Adds `project_progress` — one row per (project, checklist item) recording status (`NOT_STARTED` / `IN_PROGRESS` / `COMPLETE`), the evidence, and `verified_by` (`VALIDATION_ENGINE` / `LOG_TRIAGE` / `USER`). This is project data, never cleared by re-seeding.

Migration 0012 — blink facts. Adds `board_blink_facts` — the on-board LED pin, its clock domain, and the clock-enable hint per board, so the think-before-code checklist shows the concrete board-specific answer from the database, not from hardcoded logic.

4.2 Board profiles

A complete board profile contains: the SoC (name, vendor, architecture), the memory geometry (flash base and size, RAM base and size, and DDR for Linux-class boards), the primary storage type, the boot modes, a confidence level, a source with a citation, a list of capabilities, and a required toolchain profile.

Confidence levels. `HIGH` means the geometry came from the datasheet and is trusted. `MEDIUM` means it was entered manually (e.g. via `board add`) and has not been datasheet-verified. `LOW` means it is a skeleton — created

so a datasheet can be analyzed against it, with no geometry yet. The rule is that a board with any unknown core field can never be marked HIGH; confidence is capped, not freely chosen.

The 14 seeded boards were chosen to span the experience ladder and the common architectures, so that most beginners own one and most architectures EAEDK reasons about have a worked example:

Board	SoC / arch	Why it is in the set
STM32F103-BluePill	STM32F103C8 / Cortex-M3	The classic \$2 beginner board
Nucleo-F103RB	STM32F103RB / Cortex-M3	An official ST dev board, beginner-friendly
STM32F411RE	STM32F411RE / Cortex-M4	A step up; floating point, more memory
STM32H743	STM32H743 / Cortex-M7	High-end MCU; 512-byte vector alignment, 2 MiB flash
STM32MP157	STM32MP157 / Cortex-A7	Linux-class; the DDR-timing and U-Boot examples
Raspberry-Pi-Pico	RP2040 / Cortex-M0+	Hugely popular; external-flash/XIP boot model
WIZnet-W5500-EVB-Pico	RP2040 / Cortex-M0+	RP2040 with Ethernet; the XIP bootloader eval case
ESP32-DevKitC	ESP32 / Xtensa-LX6	Wi-Fi/BLE; the ESP-IDF and Guru-Meditation world
RTL8722DM	RTL8722DM	A less-common Wi-Fi/BLE board, for breadth
Arduino-Uno	ATmega328P / AVR	The most common hobbyist board ever made
Arduino-Mega	ATmega2560 / AVR	The larger AVR sibling
BeagleBone-Black	AM335x / Cortex-A8	Linux SBC; OTA and partition examples
Raspberry-Pi-4	BCM2711 / Cortex-A72	The most common Linux SBC
i.MX8M-Mini-EVK	i.MX8M Mini / Cortex-A53	64-bit Linux; the kernel/DTB load-address example

Onboarding a new board has three paths: the **wizard** (`eaedk board add --interactive`, with live fitment and vector-table checks as you type), **ingest** (point at a datasheet PDF; auto-creates a skeleton if the board is new), or **manual** (`eaedk board add NAME --soc --arch --flash-base ...`).

4.3 Templates

A template is a reusable project checklist for a goal type. There are **8 template types**, each versioned: `bare_metal_app` (the beginner's blink/UART first project), `bare_metal_bootloader`, `failsafe_ota`, `linux_bringup`, `linux_driver`, `low_power`, `multicore_bringup`, and `uboot_bringup`.

A **template item** has: a `key`, human-readable `text`, a `category` (e.g. `memory_layout`, `bringup`, `toolchain`), a list of `required_inputs`, and a list of `validation_rules` it maps to. For example, the `flash_fits` item requires the input `estimated_image_size` and maps to the `FLASH_CAPACITY` rule. This mapping is the link between templates and validation: the State Engine derives an item's completion from whether its mapped rules pass.

Version pinning matters because templates evolve. When you create a project, it pins the exact template version (`template_id`) it was created with. If the template is later updated, your existing project keeps the version it started on, so its checklist does not silently change underneath you. New projects get the new version. This is the same discipline as the forward-only migrations: existing work is never altered by a later change.

4.4 Seed data as YAML

All seed knowledge lives in `packages/` as YAML and is loaded by `eaedk db seed` .

```
packages/
  knowledge-seed/
    boards/          one YAML per board (14 files)
    arch_risks.yaml  architecture-family risk warnings
    board_blink_facts.yaml  LED pin + clock hint per board
    capabilities.yaml  capability → plain-language summary
    concepts.yaml     concept → anchor sentence
    debug_probes.yaml  probe → OpenOCD interface config
    driver_path.yaml  the Linux driver learning path
    eval_cases.yaml   the 14 golden eval cases
    first_mistakes.yaml  per-family beginner mistakes
    learning_path.yaml  the ordered learning steps
    learning_step_intro.yaml  what each next step introduces
    log_signatures.yaml  the 15 known-failure signatures
    risk_rules.yaml    the 4 data-driven risk rules
    soc_defaults.yaml  standard geometry per SoC
    soc_flash_profiles.yaml  SoC → OpenOCD target + default probe
  templates/         8 versioned template files
```

A board YAML (the Blue Pill, abbreviated) shows the format:

```
soc:
  name: STM32F103C8
  vendor: STMicroelectronics
  arch: arm-cortex-m3
board:
  name: STM32F103-BluePill
  flash_base: 0x08000000
  flash_bytes: 65536          # 64 KiB
  ram_base: 0x20000000
  ram_bytes: 20480           # 20 KiB SRAM
  confidence: HIGH
source:
  type: seed
  title: "STM32F103x8/xB datasheet (DS5319) + RM0008"
  uri: "https://www.st.com/resource/en/datasheet/stm32f103c8.pdf"
capabilities:
  - capability: gpio
  - capability: uart
toolchain:
  - kind: compiler
    name: arm-none-eabi-gcc
    target_triple: arm-none-eabi
    min_version: "9.0"
    severity: HIGH
    why: "Cortex-M is bare-metal ARM (Thumb); the host gcc cannot produce firmware for this MCU."
```

Why YAML and not SQL inserts. Three reasons. First, **diffability**: in a pull request, a reviewer sees exactly which fields changed, line by line, which they cannot do with a blob of `INSERT` statements. Second, **accessibility**: a contributor who knows the hardware but not SQL can add a board. Third, **review-ability of facts**: the whole project rests on facts being trustworthy, so the facts must live where humans can read and check them. Adding a new board, template, risk rule, log signature, or capability is "write a YAML file (or a block) and re-seed" — covered concretely in Chapter 12.

Chapter 5 — The Log Analysis Engine

5.1 The pipeline

When a board fails to boot, the evidence is in its log — the text it printed over the serial port, or a crash dump. The log engine reads that text and tells you what it means. The pipeline:

1. **Read the file.**
2. **Detect the format** (`parser.detect_format`): `dmesg` (Linux kernel log, recognized by `[ 1.234567]` -style timestamps), `uboot` (U-Boot bootloader output), `mcu` (bare-metal microcontroller crash — ESP32 panics, Cortex-M fault dumps), or `unknown`. Detection is by counting format-specific hints and taking the strongest.
3. **Match signatures** (`parser.match_signatures`): the text is checked against every known-failure signature whose format is compatible. A match is the deterministic, HIGH-confidence, cited answer — a known pattern mapped to a known cause and fix.
4. **The silent-boot special case**: if the capture is completely empty (the board printed nothing at all after flashing), that is a diagnosis. The engine synthesizes a match from a seeded "silent" signature explaining the common causes (wrong boot pins, failed flash, uninitialized clock or UART).
5. **On a miss, optional LLM triage**: only if nothing matched (or you passed `--deep`) and you passed `--llm`, the engine slices a 100-line window centred on the crash (`parser.crash_window`) and hands it to the post-filtered model for structural hypotheses.
6. **Write-back** (project-aware mode): when triage implicates a project gap, the finding is recorded as a tracked risk and noted on the owning checklist item (5.4).

The ordering is the whole point: a *known* failure gets a deterministic, cited answer with no model involved. The model is only ever a fallback for the unknown, and even then it cannot invent values — numbers it emits are post-filtered, except those it quotes verbatim from the log itself (which count as cited by the source).

A nice touch for MCU faults: a Cortex-M crash dump carries the faulting program-counter address. The engine turns that into a concrete next command rather than vague advice:

Find the crash location (the line of C it faulted on) with:

```
arm-none-eabi-addr2line -e build/<your-firmware>.elf 0x08001234
```

5.2 The signature database

There are 15 seeded signatures (`log_signatures.yaml`), each with a regex pattern, a format, a plain cause, a concrete fix, and a severity. The catalog:

Format	Catches	Severity
uboot	Bad Data CRC / Magic Number / Header CRC (corrupt or mis-loaded image)	HIGH
uboot	DRAM: 0 Bytes / DDR init/training/calibration failure	HIGH
uboot	SD/eMMC did not initialize (card, power, or pin-mux)	MEDIUM
dmesg	VFS: unable to mount root fs / kernel panic on rootfs	HIGH
dmesg	Kernel NULL pointer dereference / Oops / BUG (driver fault)	HIGH
uboot	PLL failed to lock (bad oscillator or multiplier/divider)	HIGH
uboot	Secure/verified boot signature or hash rejected	HIGH
mcu	ESP32 access fault — Guru Meditation StoreProhibited/LoadProhibited	HIGH
mcu	ESP32 panic — Guru Meditation / abort() / corrupt heap	HIGH
mcu	Cortex-M HardFault (escalated fault)	HIGH
mcu	Cortex-M configurable fault (CFSR: usage/bus/memory-management)	HIGH
silent	No output at all after flashing (synthesized for an empty capture)	HIGH
mcu	RTOS task stack overflow (FreeRTOS hook / Zephyr canary)	HIGH
mcu	RTOS task watchdog / task starvation	HIGH
mcu	RTOS deadlock (circular mutex dependency)	HIGH

Adding a signature means adding a block to `log_signatures.yaml` with a `format`, a `pattern_regex`, a `cause` (what went wrong, in plain language), a `fix` (what to do, concretely), and a `severity`, then re-seeding. Signatures are seeded as YAML for the same reasons as everything else — diffable, reviewable, no SQL needed — and because a good signature's value is in the *quality of its cause and fix text*, which deserves human review.

5.3 Project-aware correlation

A boot log read in isolation tells you what crashed. A boot log read *against the project's own unverified assumptions* can tell you *why*. That is what `--project-aware` adds.

Before triage, the engine builds a correlation payload (`build_correlation`) from the project: the unresolved validation checks (any FAIL or engaged-UNKNOWN), the open risks, and the **unverified hardware assumptions** — the MEDIUM/LOW-confidence facts sitting in the fact layer for this board. This is injected into the triage

prompt with an instruction to reason about how an unverified item could cause the observed failure (and, as always, not to invent values).

The canonical example is the DDR-timing scenario. A Linux board fails to boot; the log shows memory problems during relocation but matches no specific signature. Project-aware triage sees that `DDR_TIMING_VERIFIED` is an unresolved UNKNOWN for this project and that the board's DDR timing is an unverified assumption — and connects the two: the unverified DDR timing is the likely cause of the memory failure. Blind log analysis would only say "memory error somewhere"; correlation points at the specific unconfirmed decision in *your* project that explains it. The implication logic is deterministic — triage can only implicate a rule that is *already* a project gap, so it can never invent a problem you do not have.

5.4 Write-back

When project-aware triage produces a hypothesis that implicates a real project gap, the engine closes the loop (`write_back`): it appends a dated note to the checklist item(s) that own the implicated rule, and it opens (or appends to) a **tracked risk** with the severity inherited from the validation rule. The implication is keyword-based and conservative — a hypothesis only implicates a rule if its text mentions that rule's keywords *and* that rule is already a gap.

The `tracked` vs `open` status split. The `risks` table holds two very different kinds of thing, and they must not be confused:

- **Risk-engine findings** are ephemeral. They are recomputed from scratch every time you run an assessment and are never persisted as durable rows you act on. They reflect the *current* state of your inputs.
- **Tracked risks** (`status='tracked'`) are persistent. They are written by log write-back and represent a real finding from a real boot failure that you need to investigate and close.

If these shared one status namespace, every assessment would either wipe out the tracked findings (treating them as ephemeral) or the ephemeral findings would pile up as permanent rows (treating them as tracked). The split keeps "what your inputs imply right now" separate from "what a real failure told you to go fix." Tracked risks move to `status='resolved'` when you close them with `eaedk risk resolve <id> --note "..."`, which records a timestamp and your note. The resolver warns (but does not block) if you close a risk while its underlying validation is still unverified — you are allowed to overrule the engine, but you are told you are doing it. **Deduplication:** write-back upserts, so re-analyzing the same log does not pile up duplicate tracked risks for the same rule.

Chapter 6 — Getting Started

6.1 Installation

Copy-paste these one at a time. Each is explained in one sentence.

```
# 1. Download EAEDK to your computer.
git clone https://github.com/Ashut90/eaedk

# 2. Go into the folder you just downloaded.
cd eaedk

# 3. Make a private Python workspace so this install can't affect the rest of your system.
python3 -m venv .venv

# 4. Switch into that workspace (your prompt will show "(.venv)").
source .venv/bin/activate

# 5. Install EAEDK. This creates the "eaedk" command you'll use from here on.
pip install -e .

# 6. Create EAEDK's local database (one small file on your computer).
eaedk db init

# 7. Load the built-in boards, templates, and knowledge into that database.
eaedk db seed

# 8. See the boards EAEDK already knows — pick the one you have.
eaedk board list
```

Coming back later? Just `cd eaedk`, `source .venv/bin/activate`, and you are ready — steps 1–7 are one-time. For the browser interface, `pip install -e '[web]'` once, then `eaedk web`. For the optional language model, install `Ollama`, run `ollama pull qwen2.5-coder:3b`, and add `--llm` to commands that support it.

6.2 Your first project (beginner path, no hardware)

This path ends with a blinking LED in a free online simulator — no board to buy.

```
# Ask the mentor about a board. This prints what it can do, a learning path, and the exact
# next commands to run.

eaedk mentor --board STM32F103-BluePill
```

The mentor output ends with a literal recipe. Follow it:

```
# Create a project interactively. You'll be asked for a name, a board, and a goal.

eaedk project init

#   at "Project name:"   → type   blink

#   at "Select a board"  → type the number next to STM32F103-BluePill

#   at "Goal [1-9]:"    → press Enter (picks "bare-metal application – start here")
```

```
# Check the project. "feasible" means none of the checks found a blocker.

eaedk validate blink
```

A bare blink project on a seeded board is feasible immediately — the board geometry is known, and the optional checks (image size, RAM budget) are not blockers for a first build. Now export real files:

```
# Generate buildable files, including Wokwi simulator files, into a folder.

eaedk export blink --out ~/blink-fw --wokwi

# Read the guide it generated. This explains both how to simulate and how to flash.

cat ~/blink-fw/START_HERE.md
```

`START_HERE.md` offers two paths and puts **simulation first**: upload the generated `wokwi/diagram.json` and your compiled firmware to wokwi.com and watch the LED blink in your browser, with no hardware. (Wokwi natively simulates 5 of the seeded boards — the Blue Pill, Pi Pico, ESP32 DevKitC, Arduino Uno, and Arduino Mega.)

Track progress at any time:

```
# Progress is derived from evidence – what the engine verified, what you confirmed.

eaedk project status blink
```

6.3 Your first project (with physical hardware)

The flow is identical through `export`. The difference is the second path in `START_HERE.md`, which leads to a real flash and serial output.

You will need: the right cross-compiler (`arm-none-eabi-gcc` for the Blue Pill), a build system (`cmake`), a flashing tool (`openocd`), and a debug probe (an ST-Link v2 clone, the usual choice for the Blue Pill, connected over SWD). EAEDK checks what you have:

```
# Inventory the tools on your computer.

eaedk toolchain detect

# Check them against what this project's board needs, with install fixes for anything missing.

eaedk toolchain validate --project blink
```

FLASH.md (also generated by export) contains the actual flash command, filled in with the right OpenOCD target and probe for the board's SoC where EAEDK has that profile seeded — not a placeholder. Build per **START_HERE.md**, flash per **FLASH.md**, open a serial terminal at 115200 8N1, and you should see the LED blink and any debug output. If it does not, Chapter 5's log engine is the next stop.

6.4 Analyzing an unknown board's datasheet

Suppose you have a board EAEDK does not know — say an STM32F303RE — and its datasheet PDF.

```
# Ingest the datasheet. EAEDK auto-creates a skeleton board from --arch if it's new.

eaedk ingest --file stm32f303re.pdf --board STM32F303RE --arch arm-cortex-m4 --analyze
```

The **--analyze** flag prints the seven-section intelligence report (Chapter 2.7). Read it top to bottom: Section 1 is what was extracted and cited; Section 2 is the mandatory items it could *not* find and exactly where in the datasheet to look for them; Section 5 is the closest board EAEDK already knows, with what will probably carry over and what you must verify yourself.

Ask specific questions:

```
eaedk ask --board STM32F303RE "how much RAM does it have?"

eaedk ask --board STM32F303RE "what are the boot pins?"
```

The first answers from the extracted fact with a confidence and citation (or tells you honestly it was not extracted and where to search). The second returns UNKNOWN with the exact search terms, because boot-pin configuration is mandatory and the deterministic extractor never produces it.

Confirm the facts you have verified, which promotes them from staged candidates to trusted facts:

```
eaedk ingest --board STM32F303RE --review      # list the pending candidates with ids
eaedk ingest --confirm 7                      # confirm candidate #7 into the knowledge base
```

Once the geometry is confirmed, the board behaves like a seeded one: you can create a project, validate it, and export. Going from a several-hundred-page PDF to a validated project is realistically under an hour — the report tells you which few pages actually matter.

6.5 The Web UI walkthrough

`eaedk web` opens the browser interface at `http://localhost:8080`. Every tab maps to a CLI capability and the same engine.

- **Boards** — browse the seeded boards and their capabilities. Use it to pick a board and see what it can do. The output is the same capability map the mentor prints.
- **Setup** — create a project (name, board, goal). The equivalent of `project init`. Output: a created project ready to validate.
- **Validate** — run the assessment. Shows the feasibility verdict at the top (a green/yellow/red badge), then each validation result with its status and teach string, then risks. Use it before exporting. A red verdict lists the specific blockers.
- **Studio (Code Studio)** — write or paste firmware code and run the Actor-Critic review (Chapter 2.5). Shows CONFIRMED issues (real, engine-verified) separately from Advisory ones (the Critic's reasoning, not deterministically proven). Use it to harden a scaffold.
- **Datasheet (Ingest)** — upload a datasheet PDF (or paste text) for a board, including a brand-new board via the "✚ New board" option with an architecture dropdown. Shows the seven-section report and an "ask anything" box. Use it for a board EAEDK does not know.
- **Logs** — paste a boot log or crash dump. Shows the deterministic signature matches with cause and fix, and (with the model enabled) triage. Use it when a board fails to boot.
- **Mentor** — the conversational and learning-path interface. Use it to learn what to build next and to ask board-specific questions.

Each tab also prints the equivalent CLI command, so the Web UI doubles as a way to learn the CLI.

Chapter 7 — CLI Reference

Global flags (before the subcommand): `--db <path>` (use a database other than `~/.eaedk/eaedk.db`), `--json` (machine-readable output), `--llm` / `--no-llm` (enable/disable the model; default disabled). Most model-using subcommands also accept `--llm` / `--no-llm` *after* the subcommand.

Database

- `eaedk db init` — apply any pending migrations to create or upgrade the database. Run once after install. Safe to re-run; reports "up to date" when nothing is pending.
- `eaedk db seed [--force]` — load the YAML seed data. `--force` clears and reloads seed tables. Run after `db init`, and again after pulling new seed data. Misusing `--force` does not touch your project data (that lives in non-seed tables), only the seeded knowledge.

Boards

- `eaedk board list [--query TEXT]` — list known boards (name, SoC, arch). `--query` filters. Use it to find the exact board name to pass to other commands.
- `eaedk board show NAME` — full profile: geometry, DDR, storage, confidence, capabilities. Wrong name → a "not found" message that suggests the closest match and points at `board list`.
- `eaedk board add NAME --soc S --arch A [...geometry]` or `--interactive` — add a board manually (confidence MEDIUM) or via the guided wizard (live fitment checks). Missing `--soc` / `--arch` without `--interactive` is an error.
- `eaedk board fill-geometry NAME` — fill a board's missing flash/RAM from its SoC's standard geometry (the `soc_defaults` table). Use it when a board's SoC is recognized but geometry is blank. No standard on file → it tells you to ingest a datasheet instead.
- `eaedk board capability add NAME CAPABILITY` — add a capability (e.g. `UART`) so its learning-path steps unlock.

Projects

- `eaedk project init` — interactive: name, board, goal → auto-selects a template and runs an immediate assessment. The recommended way to start.
- `eaedk project new NAME --goal G [--board B]` — non-interactive project creation, for scripting.
- `eaedk project list` — all projects (name, goal, status, board).
- `eaedk project show NAME` — the full assessment plus tracked/resolved risks and recorded decisions.

- `eaedk project status NAME` — the State Engine view: progress derived from evidence, each complete item attributed to how it was proven, and the next recommended task. This is the "how am I doing?" command.
- `eaedk project archive NAME` — mark a project archived.

Inputs and checklist

- `eaedk input set NAME KEY VALUE [--confidence H/M/L] [--cite SECTION]` — set a project input (e.g. `estimated_image_size 32768`). JSON values (regions, partitions) are accepted as `{...}` / `[...]`. Confidence defaults to MEDIUM.
- `eaedk input list NAME` — list a project's inputs with confidences.
- `eaedk checklist show NAME` — the checklist with each item's mapped rule statuses.
- `eaedk checklist set NAME ITEM STATUS [--note]` — set an item to `todo` / `done` / `na` / `blocked`. Marking `done` is **refused** if a mapped rule is FAIL or an engaged UNKNOWN — you cannot mark verified-untrue work complete.
- `eaedk checklist done NAME ITEM` — the user-confirmation path for the State Engine: you confirm an item is done. Use it for items that have no automatic rule (e.g. "main loop defined").

Validation and risk

- `eaedk validate NAME [--rule RULE]` — the cited assessment: feasibility, validations (with teach strings), risks, facts, assumptions, unknowns, next step. `--rule` runs a single rule. The core command; run it before exporting.
- `eaedk risk show NAME` — live risk-engine findings, plus tracked and resolved risks.
- `eaedk risk resolve ID [--note "..."]` — close a tracked risk with a timestamp and note. Only **tracked** risks can be resolved (ephemeral findings cannot — fix the underlying validation instead). Warns if the underlying item is still unverified.

Decisions

- `eaedk decision add NAME --title T [--rationale R] [--alt JSON]` — record an engineering decision and its rationale on a project, for the project log.

Ask and explain (model-facing)

- `eaedk ask [PROJECT] [QUESTION] [--llm]` — without `--board`, prints the project assessment and (with `--llm`) a post-filtered explanation. Without `--llm` it notes the model is disabled and shows the deterministic assessment only.
- `eaedk ask --board NAME "QUESTION" [--file PDF] [--llm]` — the Board Query Engine (Chapter 2.7): a confidence-rated answer about a board. `--file` ingests a datasheet first. This is the form a beginner uses most.

- `eaedk explain NAME --rule RULE [--llm]` — explain one validation rule's result. With `--llm`, a post-filtered plain-language explanation.

Mentor

- `eaedk mentor --board NAME` — capabilities, learning path (with reasons), locked steps and how to unlock them, and the exact first-project recipe.
- `--think` — the think-before-code checklist, generated from this board's facts and goal.
- `--chat` — an interactive mentor conversation (Ctrl-D or a blank line exits).
- `--ask "QUESTION"` / `--explain CONCEPT` — a one-shot question, or an explanation of a concept (e.g. `HardFault`); the concept anchor is deterministic, with optional model elaboration.
- `--common-mistakes` — the first mistakes beginners make on this board's chip family.
- `--next [STEP]` — the next project to build; optionally name the step you just finished.
- `--review-code --project NAME` — the Actor-Critic review (Chapter 2.5). Needs the model; without it, points you at the deterministic scaffold already in your export.

Datasheet ingestion

- `eaedk ingest --file PDF --board NAME [--arch A] [--analyze] [--llm]` — extract cited fact candidates from a datasheet. A new board is auto-created from `--arch` (or it prompts). `--analyze` prints the seven-section report.
- `eaedk ingest --board NAME --review` — list pending candidates with ids.
- `eaedk ingest --confirm ID [--confidence H/M/L]` — confirm a candidate into the knowledge base. `--reject ID` — discard one. Confirmation is the only path from staged candidate to trusted fact.

Output

- `eaedk export NAME [--out DIR] [--force] [--only checklist|cmake|flash] [--wokwi]` — generate real build files. Refused if the project is not feasible (use `--force` for a clearly-labeled DRAFT). Refused if the target folder already holds a different project's files (use `--force` to overwrite). `--wokwi` adds simulator files. `--only` emits just one kind. If the board has no geometry, it warns loudly that the files will not build and offers `board fill-geometry`.

Toolchain

- `eaedk toolchain detect` — inventory the host's compilers, debuggers, flash tools, and build systems.
- `eaedk toolchain validate --project NAME` — cross-check detected tools against the board's required profile, with install fixes. Falls back to an architecture-default profile for a board with no seeded profile.

Logs

- `eaedk log analyze --file LOG [--project P] [--project-aware] [--deep] [--llm]` — analyze a boot log or crash dump. `--project-aware` correlates against the project's gaps (needs `--llm` to engage triage). `--deep` runs triage even when a signature matched, so a generic match cannot hide the root cause.

Eval and web

- `eaedk eval run [--case NAME]` — run the golden eval cases (all, or one). Exits non-zero if any fail. The regression check.
 - `eaedk web [--host H] [--port P]` — launch the Web UI (default `127.0.0.1:8080`). Tells you to `pip install -e '[web]'` if FastAPI is missing.
-

Chapter 8 — Design Decisions and Trade-offs

This chapter is the most important one for a future contributor. Each decision records what was chosen, why, and what was rejected.

LLM as a convenience layer, not the source of truth. This is the founding principle. The alternative — model first, guardrails after — is what every other AI coding tool does, and it produces confident wrong hardware facts. The cost of a wrong hardware fact is a dead board or a lost week (Chapter 1.4). So EAEDK inverts it: deterministic engines hold the truth, the model only explains. The trade-off is that EAEDK does less "magic" than a model-first tool — it will not write your whole driver — but what it *does* say about hardware is trustworthy. That trade is the product.

UNKNOWN as a hard blocker, not a soft pass. The debate (Chapter 3.3): blocking on UNKNOWN risks frustrating beginners; not blocking risks shipping a board with unverified DDR timing. The resolution was the "engaged" concept — UNKNOWN only blocks once you have started the relevant work — which keeps the discipline where it matters without nagging about untouched checks. Rejected: a global "strict mode" toggle, because a safety property you can switch off is not a safety property.

argparse over Typer. Practical reason: an early environment could not reliably install extra packages, and an offline-first tool must not fail to start because a CLI framework would not install. It stayed because "single dependency, always installable" is a permanent constraint, not a temporary one. Trade-off: the CLI code is slightly more verbose than it would be with Typer. Accepted.

SQLite over a vector database. A vector database enables semantic ("meaning-based") search and is fashionable for AI tools. EAEDK uses SQLite — a single local file — instead. The reason is the offline-first requirement: SQLite needs no server, no embedding model, no network, and ships as one file. Board matching is done with explicit, auditable column-based scoring (Chapter 2.7) rather than opaque vector similarity, which also means a beginner can see *why* two boards were called similar. Trade-off: no semantic search over the knowledge (Chapter 10); revisited in the roadmap.

Forward-only migrations. No rollback scripts. A rollback that runs against a real user's populated database risks destroying their data, and the situations a rollback is meant for are rare compared to the risk it carries. "Undo" is handled by writing a new forward migration that corrects the previous one. Trade-off: you cannot cleanly step the schema backward in development; the workaround is to delete the local dev database and re-init, which is cheap.

A single model time-sliced for Actor-Critic, not two models. The Actor and Critic roles are played by the same Ollama model called with two different system prompts (Chapter 2.5). Two separate models would mean two downloads, two resident memory footprints, and a heavier offline install — directly against the local-first goal. The

single time-sliced model gives the two-role benefit at one model's cost. Trade-off: the two roles share a model's blind spots; this is acceptable because the deterministic Arbiter, not the Critic, decides what is CONFIRMED.

The post-filter as a structural guardrail, not prompt engineering alone. It would be simpler to just tell the model "do not invent hardware facts." That is not enough: a model under the right prompt will still emit a plausible address. So the prompt says it *and* the post-filter physically removes any sentence with an uncited hardware number afterward (Chapter 2.5). Prompt engineering is advisory; the filter is enforcement. Trade-off: the filter sometimes removes a true-but-uncited sentence — accepted, because a false negative (letting an invented fact through) is far more costly than a false positive (dropping a true one).

`status='tracked'` vs `status='open'` . Without this split (Chapter 5.4), either every assessment would wipe out real findings from log triage, or ephemeral findings would accumulate as permanent rows. The split keeps "what your inputs imply right now" separate from "what a real boot failure told you to fix." It is a small schema decision that prevents a whole class of confusing behavior.

Seed data as YAML, not SQL inserts. Diffable, reviewable in a pull request line by line, and editable by a contributor who knows hardware but not SQL (Chapter 4.4). The facts the whole system trusts must live where humans can read and check them. Trade-off: a seed step is required after pulling new data; cheap and explicit.

CLI and Web UI over one engine; duplication forbidden. Two interfaces, but neither has its own logic — both call the same engine functions (Chapter 2.1). If logic were duplicated, the interfaces would drift and give different answers for the same project. The rule is enforced in review. Trade-off: the Web layer is constrained to thin wrappers and cannot take shortcuts; accepted, because correctness across interfaces matters more than Web-side convenience.

Wokwi-first export. The generated `START_HERE.md` puts the simulation path before the physical-hardware path (Chapter 2.3). The reason is the Level-0 beginner: requiring hardware purchase, wiring, and a debug probe before the first success loses people. Simulation gives a blinking LED in a browser in minutes, which is the moment that makes someone continue. The physical path is right below it for when they are ready. Trade-off: simulation is limited to the 5 Wokwi-supported boards (Chapter 10); for others the guide leads with the physical path.

Chapter 9 — Testing and Validation

9.1 The test suite

The suite is **263 passing tests** across 22 test files (`core/tests/`), and it runs entirely offline — the LLM tests use a fake in-memory provider, so no Ollama is required to run the full suite.

```
PYTHONPATH=core python3 -m pytest -q
```

By category:

- **Unit tests** — individual pure functions: the validation rules (`test_engines.py`), the risk DSL parser, the post-filter (`test_llm.py`), the log parser (`test_log_engine.py`), the extractor (`test_ingest.py`), the similarity scorer, the state engine (`test_state_engine.py`).
- **Integration tests** — multi-engine flows: project init → validate → export (`test_project_init.py` , `test_output.py` , `test_bare_metal_app.py`), onboarding (`test_onboard.py` , `test_onramp.py`), the mentor layer (`test_mentor.py` , `test_mentor_complete.py` , `test_mentor_ux.py`).
- **Golden eval cases** — the 14 trust-critical scenarios run through the real engines (9.2).
- **Web tests** (`test_web.py`) — the FastAPI routes, confirming they call the same engine and return the expected structure.
- **LLM tests** (`test_llm.py` , `test_multiagent_fix.py`) — the gateway, post-filter, and Actor-Critic with a fake provider, so the trust guarantees are tested without a real model.

The suite is structured so that the deterministic core is tested independently of the model: you can verify every trust property — that an invented address is stripped, that UNKNOWN blocks, that the Arbiter overrules the Critic — with no network and no Ollama.

9.2 The eval harness

The 14 golden eval cases (`eval_cases.yaml`), each asserting a subset of the result (Chapter 3.5):

Case	Asserts	Why it exists
<code>stm32f411_bootloader_fits</code>	feasible; FLASH/VECTOR/OVERLAP/RAM all PASS; no FLASH_TIGHT	A clean bootloader baseline
<code>stm32f411_image_overflow</code>	not_feasible; FLASH_CAPACITY FAIL; FLASH_TIGHT fires	An image too big for flash must block
<code>uboot_ddr_unverified_is_unknown</code>	blocked ; DDR_TIMING_VERIFIED UNKNOWN; DDR_GUESSED risk; "DDR timing" in unknowns	The trust-critical case: unverified DDR must block, not pass
<code>ota_ab_asymmetry_fails</code>	not_feasible; PARTITION_AB_SYMMETRY FAIL	Mismatched OTA slots must fail
<code>linux_kernel_load_in_ddr_init_region_fails</code>	not_feasible; LOAD_ADDR_CONFLICT FAIL	A kernel loaded over the DDR init region must fail
<code>rp2040_w5500_xip_bootloader_fits</code>	feasible; all memory rules PASS	The RP2040 external-flash/XIP case: a bootloader in external QSPI flash, executed in place, validates correctly
<code>stm32h743_bootloader_fits</code>	feasible	An M7 (512-byte vector alignment, 2 MiB flash) baseline
<code>pinmux_conflict_detected</code>	not_feasible; PINMUX_CONFLICT FAIL	A pin assigned to two signals must fail
<code>pinmux_clean_passes</code>	feasible; PINMUX_CONFLICT PASS	The clean counterpart, so the rule is not over-eager
<code>power_sequence_out_of_order_fails</code>	not_feasible; POWER_SEQUENCE FAIL	IO powered before core must fail
<code>power_sequence_valid_passes</code>	feasible; POWER_SEQUENCE PASS	The clean counterpart
<code>secure_boot_chain_complete_passes</code>	feasible; all four SECURE_BOOT rules PASS	A complete secure-boot chain validates
<code>secure_boot_debug_open_fails</code>	not_feasible; SECURE_BOOT_DEBUG_LOCKED FAIL	Debug left open in production must fail
<code>secure_boot_key_storage_unrecognised_blocks</code>	blocked ; SECURE_BOOT_KEY_STORAGE UNKNOWN; "key storage" in unknowns	A key in rewritable storage is an engaged UNKNOWN → blocked

Two of these deserve emphasis. `uboot_ddr_unverified_is_unknown` is the case that encodes the whole philosophy: it would be easy to make unverified DDR "pass," and this case is the wall that stops that regression forever. `rp2040_w5500_xip_bootloader_fits` guards a subtle real-world correctness point: the RP2040 runs code from external QSPI flash mapped into the address space and executed in place (XIP), so its flash base is `0x10000000`

rather than an STM32's `0x08000000` , and the memory rules must validate that layout correctly rather than assume an internal-flash model.

9.3 The dogfood methodology

"Dogfooding" means using your own product as a real user would. EAEDK was tested by running it as a genuine zero-experience beginner, and by running it against a real datasheet for a board not in the database.

The most valuable findings came from these sessions, not from unit tests. The real-hardware dogfood (`docs/10-dogfood-findings.md`) drove the mentor layer's existence — a beginner with a working tool still did not know *what to do with it*, which is why the capability maps, learning paths, and think-before-code checklists were built. A later dogfood on an STM32F303RE datasheet found five concrete gaps — a board EAEDK did not know was a dead end with no on-ramp; the extractor missed facts split across prose; the query engine could not tell "not ingested" from "ingested but not extracted" — each of which became a fix (`docs/20-datasheet-fixes.md`).

Why dogfooding is the most valuable test for a tool like this: the unit tests verify that the engines do what the engineers intended, but only a real beginner reveals where the *intentions themselves* were wrong — where a correct answer was given in a form a beginner could not use, or where a dead end existed that no rule was checking for. A trust-and-teaching tool fails in ways that are about the human experience, and those failures are invisible to a test that only checks return values.

9.4 How to add tests

- **A golden eval case:** add to `eval_cases.yaml` (name, goal type, inputs with a seeded `board` , and an `expected` block asserting only the feasibility/validations/risks/unknowns the case is about), re-seed, run `eaedk eval run --case <name>` . Keep it small — one case, one point.
 - **A unit test:** follow the existing patterns in `core/tests/` . Pure functions (a rule, the post-filter) are tested directly with a constructed context and an asserted result — no database needed. Database-backed flows use the in-memory-seeded connection helper the existing tests use.
 - **Testing a new validation rule:** add a unit test that calls the rule's function with a context that should PASS, one that should FAIL, and one that should be UNKNOWN (missing inputs), asserting each. Then add at least one golden eval case that exercises it end to end through the orchestrator. The rule is not done without both (Chapter 12).
 - **Testing a new log signature:** add a test that runs `analyze_log` over a small log containing the failure text and asserts that your signature matched with the right cause — the existing `test_log_engine.py` shows the shape.
-

Chapter 10 — Limitations and Known Gaps

Stated plainly. For each: what it means in practice, whether it is planned, and today's workaround.

- **The PDF extractor is line-bounded.** It splits pages into sentences and scans them, so a fact split across a page break, or laid out in a multi-column page, can be missed. *In practice:* clean prose and memory-map tables extract well; awkward layouts leave gaps. *Planned:* incremental improvement, not a rewrite. *Workaround:* the report's Section 2 tells you exactly which mandatory items were not found and where to look, and `--review` lets you confirm anything you find by hand — the gap is surfaced honestly rather than hidden.
- **The Actor-Critic Critic can emit advisory noise.** The small (3B) model sometimes raises weak advisory claims about inputs you have not set. *In practice:* you may see Advisory items that are not real problems. *Planned:* prompt and model improvements. *Workaround:* the CONFIRMED/Advisory split exists precisely for this — only CONFIRMED items are engine-verified; Advisory ones are explicitly "the Critic's reasoning, not deterministically proven," so they inform without misleading.
- **Wokwi support is limited to 5 boards** (Blue Pill, Pi Pico, ESP32 DevKitC, Arduino Uno, Arduino Mega). *In practice:* other boards get a clear note instead of simulation files. *Planned:* tied to Wokwi's own catalogue; expands as Wokwi does. *Workaround:* for an unsupported board, follow the physical-hardware path in `START_HERE.md`.
- **No vector database / no semantic search.** Board similarity uses explicit column-based scoring (arch, flash, RAM, vendor, peripherals — Chapter 2.7), not meaning-based search over the knowledge. *In practice:* you cannot ask a free-text question and have EAEDK semantically search all board knowledge. *Planned:* yes (Chapter 11), as an *additive* layer that must not become a new source of unverified truth. *Workaround:* the deterministic scoring is auditable and sufficient for the "closest known board" use case it serves.
- **No desktop UI.** CLI and Web UI only. *In practice:* no native app. *Planned:* under consideration (Chapter 11). *Workaround:* the Web UI is the point-and-click option and runs locally.
- **No real-time probe integration.** EAEDK cannot detect which debugger is physically plugged into your computer. *In practice:* it generates the flash command from the board's SoC profile and the probe a beginner most likely owns, but it does not confirm your actual probe. *Planned:* yes (Chapter 11). *Workaround:* `FLASH.md` covers the common probe for the board, and `toolchain detect` confirms the flashing *software* is installed.
- **Log analysis requires text output.** Binary diagnostic protocols are not parsed. *In practice:* only logs that are human-readable text (serial console output, dmesg) are analyzed. *Planned:* not near-term. *Workaround:* capture your board's serial output as text, which is how most bring-up debugging is done anyway.

- **Datasheet ingestion is English-only.** Extraction and the report are built around English datasheet conventions and have not been tested on other languages. *In practice:* a non-English datasheet may extract poorly. *Planned:* not near-term. *Workaround:* most vendor datasheets are available in English; use the English edition.

The honest summary: EAEDK is strong at deterministic checking and honest about what it does not know. Its gaps are mostly at the edges (messy PDFs, unsupported simulation boards, no live hardware detection), and in every case the design surfaces the gap rather than papering over it with a guess.

Chapter 11 — Roadmap

Planned work, with what each enables, what it depends on, and why it is deferred.

- **Vector database integration (semantic search over board knowledge).** *Enables:* free-text questions answered by meaning-based search across all board and concept knowledge. *Depends on:* an embedding model that can run locally without breaking the offline guarantee, and a design that keeps semantic results clearly lower-trust than cited facts. *Deferred because:* the offline-first constraint makes the dependency non-trivial, and it must be additive — it cannot become a back door for unverified "facts."
- **Desktop UI (Electron or Tauri).** *Enables:* a native installable app. *Depends on:* a packaging story that does not pull in a heavy toolchain. *Deferred because:* the Web UI already covers point-and-click, so this is convenience, not capability.
- **Real-time probe integration (auto-detect the plugged-in debugger).** *Enables:* a flash command tailored to your *actual* probe, and detection of a missing or misconfigured probe. *Depends on:* USB device enumeration across operating systems. *Deferred because:* it adds OS-specific code and the current SoC-profile approach already produces a working command for common setups.
- **Errata tracking (known silicon bugs by board revision).** *Enables:* warnings about documented chip bugs relevant to your exact silicon revision. *Depends on:* a curated, cited errata dataset (the `errata` source type already exists in the schema). *Deferred because:* the value is entirely in data quality, and seeding accurate, cited errata is a large, careful effort.
- **Community board profiles (submit and share board YAML).** *Enables:* a shared library of board profiles contributed by users. *Depends on:* a review and confidence-assignment process so community data does not silently become HIGH-confidence truth. *Deferred because:* the trust model must be preserved — community facts need clear provenance and a review gate before they are trusted.
- **CI/CD plugin (run validation rules on every git push).** *Enables:* EAEDK's checks as an automated gate in a team's pipeline. *Depends on:* a stable machine-readable output (the `--json` flag is the foundation) and a thin CI wrapper. *Deferred because:* it is straightforward once the rule set and JSON output are stable, so it follows rather than leads.
- **Silicon-vendor partnerships (pre-seeded profiles for new eval boards).** *Enables:* accurate, vendor-blessed profiles for new boards at launch. *Depends on:* relationships and a contribution process. *Deferred because:* it is an organizational effort, not an engineering one, and benefits most once the community-profile pipeline exists.

The roadmap's through-line: every item must preserve the trust model. Nothing on this list is allowed to turn the model into a source of unverified hardware truth or to break the offline-first guarantee.

Chapter 12 — Contributing

EAEDK is built to be extended through diffable YAML and small, well-tested additions. The most common contributions:

Add a new board. Create `packages/knowledge-seed/boards/<name>.yaml` following the format in Chapter 4.4: a `soc` block (name, vendor, arch), a `board` block (geometry, confidence), a `source` block (title, uri — the citation), a `capabilities` list, and a `toolchain` profile. Set `confidence: HIGH` only if the geometry is datasheet-verified; otherwise MEDIUM. Re-seed and confirm with `eaedk board show <name>`.

Add a new validation rule. Add a function to `engines/validation/rules.py` decorated with `@rule(key, goals, user_inputs, severity)`, returning a `ValidationResult` of PASS/FAIL/UNKNOWN. Add a `RULE_TEACH` entry — a beginner contributor must never be left with a bare rule name, so the teach string (what the field is, units, where to find it, the consequence) is required. Map the rule into the relevant template item(s) via `validation_rules`. **A new rule is not complete without a golden eval case** that exercises it (Chapter 9.4), and a unit test covering its PASS/FAIL/UNKNOWN branches.

Add a new log signature. Add a block to `log_signatures.yaml` with `format`, `pattern_regex`, `cause` (plain language), `fix` (concrete action), and `severity`. The teach value is in the cause and fix text — write them for a beginner who has never seen this failure. Add a test that runs `analyze_log` over a sample log and asserts the match.

Add a new template. Create `packages/templates/<key>.v<N>.yaml` with `key`, `name`, `version`, `goal_type`, and an `items` list. Each item needs a `key`, `text`, `category`, `required_inputs`, and the `validation_rules` it maps to. Mapping items to rules is what lets the State Engine derive completion from evidence.

Branch hygiene. One feature branch per item. Merge with fast-forward only (`--ff-only`), so history stays linear. Commit messages and pull-request bodies must not contain AI-assistant attribution. Tag releases.

The preservation rule. Changes are **additive**: existing tests must all still pass, and existing behavior (CLI verbs, schema, seed data, rules, templates) must not change unless that is the explicit point of the change. The subset-matching eval design (Chapter 3.5) and forward-only migrations (Chapter 8) exist precisely so that adding new capability does not break old guarantees. Before opening a pull request, run the full suite and the eval harness:

```
PYTHONPATH=core python3 -m pytest -q
eaedk eval run
```

Both must be green. A contribution that adds capability while keeping every existing test and eval case passing is exactly the shape EAEDK is built to accept.

Reason first. Build second. Verify always.